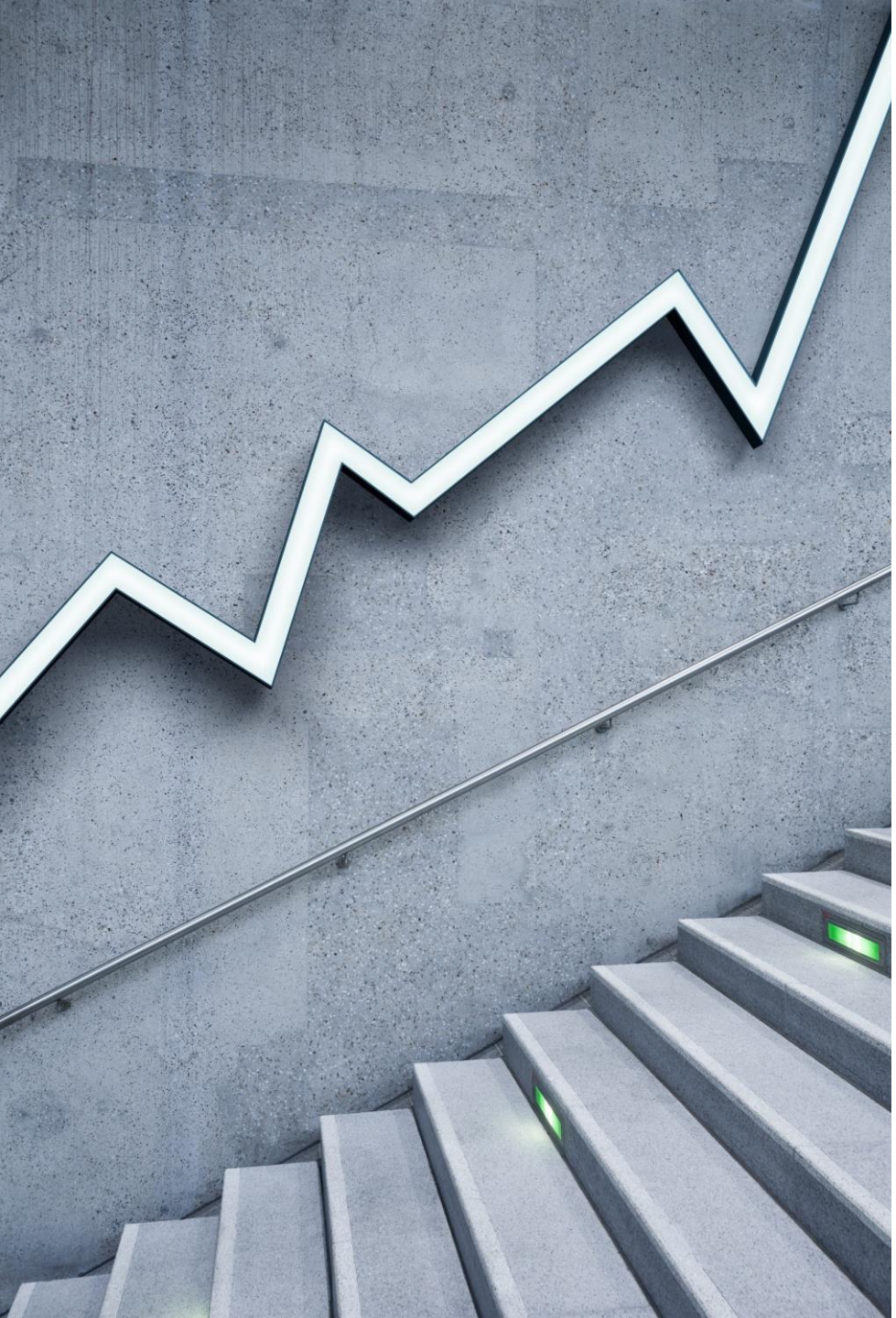


Database Laravel

WEB LANJUT MVC

1. Pengantar Laravel

Instalasi

```bash
composer create-project --prefer-dist laravel/laravel nama_proyek
```

Konfigurasi Laravel

```bash
cp .env.example .env
```

Struktur Folder dan Arsitektur Laravel

Fitur Utama Laravel:

Routing yang Sederhana: Memudahkan dalam mendefinisikan URL dan menangani permintaan HTTP.

ORM Eloquent: Mempermudah interaksi dengan database menggunakan paradigma Object-Relational Mapping.

Middleware: Menyediakan cara untuk memfilter dan memanipulasi permintaan HTTP.

Sistem Templating Blade: Memungkinkan penggunaan template dengan sintaks yang bersih dan efisien.

Keamanan: Fitur bawaan untuk melindungi aplikasi dari serangan umum seperti SQL injection dan CSRF.

2. Database Migration

Membuat Migration

- php artisan make:migration create_users_table

Mengelola Migration

- php artisan migrate

Rollback dan Reset Migration

- php artisan migrate:rollback
- php artisan migrate:reset

3. Eloquent ORM

Eloquent adalah ORM (Object-Relational Mapping)

- Eloquent adalah ORM (Object-Relational Mapping) yang disediakan oleh Laravel
- Setiap model berhubungan dengan tabel di database
- Eloquent juga mendukung hubungan antar tabel, seperti one-to-many dan many-to-many

Membuat Model dan Menghubungkan ke Tabel

- php artisan make:model NamaModel

```php
class User extends Model {
    protected $table = 'nama_tabel';
}
```

Hubungan Lanjutan: One-to-Many, Many-to-Many, Polymorphic

One-to-Many

```php
public function posts() {
    return $this->hasMany(Post::class);
}
```

Many-to-Many

```php
public function roles() {
    return $this->belongsToMany(Role::class);
}
```

4. Query Builder

Mengambil Data

- $users = DB::table('users')->get();
- $activeUsers = DB::table('users')->where('active', 1)->get();
- $user = DB::table('users')->where('id', 1)->first();

Menambahkan Data

- DB::table('users')->insert(['name' => 'John Doe',   'email' => 'john@example.com',]);

Mengupdate dan Menghapus Data

- DB::table('users')->where('id', 1)->update(['active' => 0]);
- DB::table('users')->where('id', 1)->delete();

Query Lanjutan: Aggregates, Joins, Subqueries

Aggregates,

- $userCount = DB::table('users')->count();
- $averageAge = DB::table('users')->avg('age');

Joins

- $usersWithPosts = DB::table('users') ->join('posts', 'users.id', '=', 'posts.user_id') ->select('users.name', 'posts.title') ->get();
- $usersWithPosts = DB::table('users') ->leftJoin('posts', 'users.id', '=', 'posts.user_id') ->select('users.name', 'posts.title') ->get();

Subqueries

- $recentUsers = DB::table('users') ->where('created_at', '>', function ($query) { $query->select(DB::raw('DATE_SUB(NOW(), INTERVAL 1 MONTH')); }) ->get();

Optimasi Query Menggunakan Query Builder

Menggunakan Indeks:

Menghindari Select All:

- $users = DB::table('users')->select('name', 'email')->get();

Batch Insert:

- DB::table('users')->insert(
- [['name' => 'John Doe', 'email' => 'john@example.com'],
  - ['name' => 'Jane Doe', 'email' => 'jane@example.com'],]);

Eager Loading:

- $users = User::with('posts')->get();

5. Validasi Data

Apa itu Form Request?

Membuat Form Request

- **php artisan make:request StoreUserRequest**
- app/Http/Requests/StoreUserRequest.php

Menggunakan Form Request di Controller

```php
public function rules() {
    return [
        'name' => 'required|string|max:255',
        'email' => 'required|email|unique:users,email',
        'password' => 'required|string|min:8|confirmed',
    ];
}
```

Pesan Error dan Penanganan Validasi

Pesan Error

Penanganan Validasi

php Copy code

```php
public function messages() {
    return [
        'name.required' => 'Nama diperlukan.',
        'email.required' => 'Email diperlukan.',
        'password.min' => 'Password minimal harus 8 karakter.',
    ];
}
```

Penanganan Validasi

Laravel secara otomatis mengalihkan pengguna kembali ke halaman sebelumnya dengan pesan kesalahan validasi. Namun, Anda dapat menyesuaikan penanganan ini di metode `failedValidation()` dalam Form Request:

php Copy code

```php
protected function failedValidation(Validator $validator) {
    throw new HttpResponseException(response()->json([
        'errors' => $validator->errors(),
    ], 422));
}
```

Validasi Lanjutan: Menggunakan Rule Kustom

Apa itu Rule Kustom?

Membuat Rule Kustom

- php artisan make:rule Uppercase

Menggunakan Rule Kustom dalam Form Req

Setelah itu, Anda bisa menambahkan logika validasi di dalam metode `passes()`:

php — Copy code

```php
public function passes($attribute, $value) {
    return strtoupper($value) === $value;
}

public function message() {
    return 'The :attribute must be uppercase.';
}
```

Menggunakan Rule Kustom dalam Form Request

Setelah membuat rule kustom, Anda dapat menggunakannya di Form Request:

php — Copy code

```php
public function rules() {
    return [
        'name' => ['required', 'string', 'max:255', new Uppercase],
    ];
}
```

6. Seeding dan Factory

Pengertian dan Manfaat Database Seeding

Manfaat Database Seeding

1. **Pengujian Aplikasi:**
2. **Mempercepat Pengembangan**
3. **Menjaga Konsistensi Data:**
4. **Mendemonstrasikan Fitur:**

Membuat Factory untuk Generasi Data Dummy

Apa itu Factory?

Membuat Factory

- **Buat Factory Baru:**
  - php artisan make:factory UserFactory
- **Definisikan Struktur Data:**

Menggunakan Factory

- // Membuat satu pengguna
- $user = User::factory()->create();
- // Membuat sepuluh pengguna
- $users = User::factory()->count(10)->create();

```php
use App\Models\User;
use Illuminate\Database\Eloquent\Factories\Factory;

class UserFactory extends Factory {
    protected $model = User::class;

    public function definition() {
        return [
            'name' => $this->faker->name(),
            'email' => $this->faker->unique()->safeEmail(),
            'password' => bcrypt('password'), // Enkripsi password
        ];
    }
}
```

Menggunakan Seeder untuk Pengisian Data Awal

Apa itu Seeder?

Membuat Seeder

- **Buat Seeder Bar**
  - php artisan make:seeder UserSeeder
- Definisikan Data yang Akan Dimasukkan: Buka file

Menjalankan Seeder

- php artisan db:seed --class=UserSeeder
- php artisan db:seed

Menyisipkan Seeder di DatabaseSeede

```php
use Illuminate\Database\Seeder;
use App\Models\User;

class UserSeeder extends Seeder {
    public function run() {
        // Menggunakan factory untuk membuat 10 pengguna
        User::factory()->count(10)->create();
    }
}
```

```php
public function run() {
    $this->call(UserSeeder::class);
}
```

7. Relasi dan Pengambilan Data

Apa itu Eager Loading?

- $users = User::with('posts')->get();

Apa itu Lazy Loading?

- $user = User::find(1);
- $posts = $user->posts;
- // Ini akan men-trigger query untuk mengambil postingan

Penggunaan Scopes untuk Query yang Reusable

Apa itu Scopes?

Membuat Scope

Menggunakan Scope

- $activeUsers = User::active()->get();
- $activeUsersWithPosts = User::active()->with('posts')->get();

```php
class User extends Model {
    public function scopeActive($query) {
        return $query->where('active', 1);
    }
}
```

Keuntungan Menggunakan Scopes
1.Reusable:
2.Membaca Lebih Mudah:
3.Organisasi yang Lebih Baik:

8. Database Transaction

```php
public function store(StoreTaskRequest $request)
{
    DB::transaction(function () use ($request) {
        $task = Task::create($request->validated());
        // Lakukan operasi lain yang terkait
    });
}
```

9. Pengelolaan Database di Cloud

Daftar AIVEN -> pilih service gratis mysql

10. Keamanan Database

1. Penggunaan Parameter Binding
2. Validasi Input
3. Menggunakan ORM (Eloquent)
4. Menghindari Query Raw (jika harus maka gunakan binding)
5. Pembaruan berkala (update)

```php
// Menggunakan Query Builder
$user = DB::table('users')->where('email', '=', $email)->first();

// Menggunakan Eloquent
$user = User::where('email', $email)->first();
```

```php
$request->validate([
    'email' => 'required|email',
    'password' => 'required|min:8',
]);
```

```php
DB::select('SELECT * FROM users WHERE email = ?', [$email]);
```

Praktikum

Silahkan ke Modul

11. Caching Database

Polymorphic